

Forward Deployed Engineer Task

Objective: Build a minimal, scalable compliance layer for internal AI-powered apps.

Teams at a company are starting to build small internal tools, several of which call an LLM. Today, every submission gets manually reviewed by one person checking auth, secrets, LLM-safety issues, and whether the app is observable enough to catch problems before a user reports them. That doesn't scale past a handful of apps, and it doesn't account for the fact that some apps stay owned by the team that built them, while others eventually get handed off to a central team to run.

Design and build a lightweight foundation for this.

AI tools welcome

Same as always. Approach section required. We ask how you used AI tools, not whether you did.

Requirements

1. Minimal reference app Your choice, calls an LLM for one narrow purpose. Keep it trivial; almost all your effort should go into #2 and #3, not app features. Basic auth in front of it is enough.

2. Compliance bar and automated checker Define what counts as “safe to ship” for an app like this, and defend that definition. Build something that checks your reference app against your own bar automatically, such as a script, CI step, or LLM-as-judge tool. It should also be able to run against a different app, not be hardcoded to only understand the one you built.

3. Design write-up 1–3 pages covering:

- What's in your bar and why. What you deliberately left out, and why.
- Some teams will keep building and maintaining their own app after submitting it. Others will effectively hand it off for a central team to own going forward. How does your design facilitate both without ending up with two divergent sets of tooling? What does the way of working look like differently for each track: who owns monitoring alerts, feedback routing, deploys, rollbacks, and what infrastructure/tooling decisions make that split practical rather than chaotic?
- How this holds up at 50 teams submitting concurrently: where it bends first, what you'd change.
- Which findings should stay fully automated vs. always route to a human, and why you drew the line there.
- What you'd tackle first with two more weeks.

4. Documentation README with prerequisites, run instructions, and Approach section.

5. Submission Public repo, visible commit history, and link shared.

How we will test

1. Run the README steps.
2. Run their checker against their own reference app.
3. We hand them a second small app we've prepared live, with a couple of deliberate issues, and see what their checker catches and misses.
4. 45–60 min walkthrough. We'll treat the write-up like a real design review, and push specifically on the ownership-split answer (how it changes for a developer-owned vs. centrally-owned app) and the hallucination/scale reasoning.

Note on scope

No tight time box, but this should be a few focused hours, not a finished platform. We're evaluating judgment and design instinct.